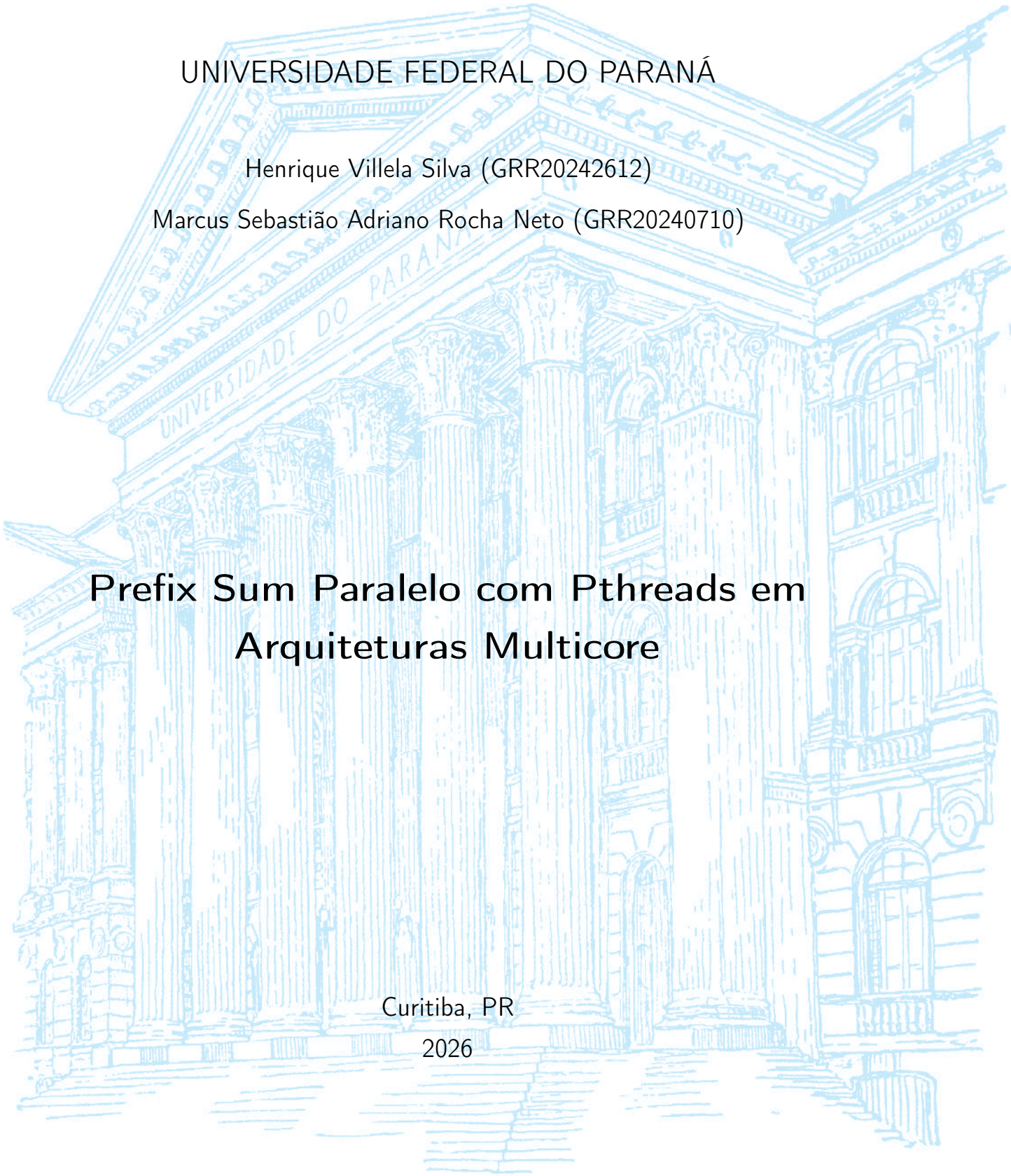
UNIVERSIDADE FEDERAL DO PARANÁ

Henrique Villela Silva (GRR20242612)

Marcus Sebastião Adriano Rocha Neto (GRR20240710)



Prefix Sum Paralelo com Pthreads em Arquiteturas Multicore

Curitiba, PR

2026

Henrique Villela Silva (GRR20242612)
Marcus Sebastião Adriano Rocha Neto (GRR20240710)

Prefix Sum Paralelo com Pthreads em Arquiteturas Multicore

Relatório técnico apresentado como requisito parcial para avaliação na disciplina de Programação Paralela (CI1316), Departamento de Informática, Universidade Federal do Paraná.

Universidade Federal do Paraná (UFPR)
Setor de Ciências Exatas
Departamento de Informática (DInF)

Orientador: Prof. Dr. Wagner M. Nunan Zola

Curitiba, PR
2026

RESUMO

Este relatório apresenta a implementação e a análise experimental do algoritmo paralelo de *Prefix Sum* Exclusivo (*in-place*) sobre um vetor de 8.000.000 de elementos do tipo `long long` (64 MB), utilizando a biblioteca POSIX Threads (`pthreads`) em linguagem C. O projeto foi estruturado com um *Pool de Threads* persistente com afinidade estática de núcleos (*core pinning*), sincronização otimizada com apenas duas barreiras por chamada utilizando controle atômico em C11, e divisão em blocos (*chunks*) dimensionados para caber na memória cache L2. Os testes foram executados no nó `wz10` do cluster da disciplina, composto por dois processadores Intel Xeon E5410 (8 núcleos físicos no total e 24 MB de cache L2). A estratégia de particionamento em chunks (chunkSize = 196.608) gerou um ganho de vazão de até +52,48% (1,52×) sobre a versão sem chunks, atingindo vazão média máxima de 494,48 MOPS com 4 threads e pico individual de 517,92 MOPS com 8 threads.

Palavras-chave: Programação Paralela. POSIX Threads. Prefix Sum. Cache L2. Multicore. SLURM.

ABSTRACT

This report presents the implementation and experimental evaluation of an exclusive in-place parallel Prefix Sum algorithm over an 8,000,000 `long long` array (64 MB) using POSIX Threads (`pthreads`) in C. The implementation employs a persistent thread pool with static core affinity (core pinning), an optimized synchronization scheme using only two barriers per function call via C11 atomic operations, and a chunked partitioning strategy designed for L2 cache reuse. Experiments were conducted on cluster node `wz10`, equipped with two Intel Xeon E5410 processors (8 physical cores in total and 24 MB L2 cache). The chunked strategy (chunkSize = 196,608) yielded a throughput improvement of up to +52.48% (1.52×) compared to the non-chunked baseline, achieving a peak average throughput of 494.48 MOPS with 4 threads and an individual peak of 517.92 MOPS with 8 threads.

Keywords: Parallel Programming. POSIX Threads. Prefix Sum. L2 Cache. Multicore. SLURM.

SUMÁRIO

1 IMPLEMENTAÇÃO E ALGORITMO PARALELO 7
1.1 Pool de Threads e Afinidade de Cores 7
1.2 Algoritmo em 2 Fases 7
1.3 Otimização de Sincronização: Duas Barreiras e Sinalização Atômica 8
1.4 Processamento em Chunks para Reúso da Cache L2 9

2 ARQUITETURA DO PROCESSADOR E TOPOLOGIA . . 11
2.1 Características do Processador (wz10) 11
2.2 Hierarquia de Memória Cache 11
2.3 Relação entre a Arquitetura e os Resultados 13

3 METODOLOGIA EXPERIMENTAL E MEDIÇÃO 15
3.1 Desconto do Tempo de Cópia (memcpy) 15
3.2 Experiência 0 — Escolha do chunkSize 15

4 RESULTADOS EXPERIMENTAIS E ANÁLISE DE DESEMPENHO 17
4.1 Experiência A — Com Chunks L2 (chunkSize $= 196.608$) . . 17
4.2 Experiência B — Sem Chunks (chunkSize $= 0$) 17
4.3 Comparativo Direto: Experiência A vs. Experiência B . . . 19
4.4 Discussão dos Resultados 19

5 INSTRUÇÕES DE COMPILAÇÃO, EXECUÇÃO E ARQUIVOS ENTREGUES 21
5.1 Como Compilar o Projeto 21
5.2 Como Executar o Programa 21
5.3 Como Submeter os Experimentos via SLURM 22
5.4 Estrutura dos Arquivos Entregues 22

6 CONCLUSÃO 25

APÊNDICES 27

APÊNDICE A – SAÍDA COMPLETA DO COMANDO LSCPU NO NÓ WZ10 29

1 IMPLEMENTAÇÃO E ALGORITMO PARALELO

O objetivo deste trabalho é implementar de forma eficiente o cálculo de **Prefix Sum Exclusivo** sobre um vetor contíguo de $N = 8.000.000$ elementos do tipo `long long` (64 MB de memória), operando de maneira *in-place* (ou seja, sobrescrevendo os valores no próprio vetor de entrada `Vec`).

1.1 POOL DE THREADS E AFINIDADE DE CORES

Para não ter o custo de criar (`pthread_create`) e finalizar (`pthread_join`) threads a cada chamada da função durante as medições ($NTIMES = 100$), implementamos um **Pool de Threads** estático na função `initThreadPool`:

- As $nThreads - 1$ threads trabalhadoras (`workerKernel`) são criadas apenas uma vez, antes do início das medições. A thread principal (`main`) atua diretamente como a trabalhadora de índice $id = 0$.
- Cada thread é fixada em um núcleo físico específico por meio de `pthread_setaffinity_np` e da macro `CPU_SET(id, &cpuset)`. Isso evita que o sistema operacional mude as threads de núcleo durante a execução, mantendo os dados aquecidos nas caches L1 e L2 de cada núcleo.

1.2 ALGORITMO EM 2 FASES

Para cada bloco de dados a ser processado, o trabalho é dividido em fatias contíguas $[start, end)$ entre as $nThreads$ threads, executando em duas fases:

1. **Fase 1 (Soma Local):** Cada thread id percorre a sua fatia somando os elementos e salva a soma total em `PartialSum[id]`. Nesta fase o vetor original é apenas lido, sem nenhuma alteração.

2. **Barreira Algorítmica (`algo_barrier`):** Sincroniza todas as threads para garantir que todos os valores em `PartialSum` estejam calculados antes de iniciar a próxima fase.

3. **Fase 2 (Acumulação e Escrita In-Place):** Cada thread *id* soma os valores de `PartialSum[0 ...id-1]` (mais o acumulador de blocos anteriores, `chunkBase`) para descobrir o prefixo inicial da sua fatia. Depois, percorre sua fatia $[start, end)$ substituindo cada `Vec[i]` pela soma acumulada até o elemento anterior.

1.3 OTIMIZAÇÃO DE SINCRONIZAÇÃO: DUAS BARREIRAS E SINALIZAÇÃO ATÔMICA

Em uma implementação convencional com pool de threads, seriam necessárias três barreiras por chamada:

1. Uma barreira na entrada da função para liberar as threads trabalhadoras do pool;

2. Uma barreira no meio (entre a Fase 1 e a Fase 2);

3. Uma terceira barreira no final da Fase 2 para que a thread `main` espere todas as threads terminarem antes de retornar.

No entanto, o uso excessivo de barreiras POSIX (`pthread_barrier_wait`) impõe sobrecarga de sincronização e contenção desnecessária. Para otimizar o fluxo e manter o algoritmo com rigorosamente **apenas duas barreiras por chamada**, substituímos a barreira final de encerramento por uma sinalização leve com **contador atômico em C11** (`atomic_int threads_completed`):

- **1ª Barreira (`pool_barrier`):** Sincroniza a thread `main` com as trabalhadoras para iniciarem a execução do algoritmo juntas.

- **2ª Barreira (`algo_barrier`):** Sincroniza a passagem entre a Fase 1 e a Fase 2.

- **Término sem 3ª Barreira:** Ao terminar a Fase 2, cada trabalhadora executa `atomic_fetch_add(&threads_completed, 1)` e já retorna diretamente para o início do seu laço, ficando posicionada na `pool_barrier` para a próxima iteração.

- A thread `main` (que atua como a thread 0) faz apenas uma espera atômica em memória (`while (atomic_load(&threads_completed) < nThreads - 1)`), que é satisfeita assim que a última trabalhadora conclui sua escrita.

Com esse desenho, o algoritmo executa exatamente duas barreiras por chamada a `ParallelPrefixSumPth`, reduzindo a latência de sincronização no retorno da função.

1.4 PROCESSAMENTO EM CHUNKS PARA REÚSO DA CACHE L2

Como o algoritmo precisa ler o vetor na Fase 1 e percorrê-lo de novo na Fase 2 para escrever os resultados, processar o vetor inteiro de 64 MB de uma vez só (chunkSize = 0) faz com que os dados lidos no começo da Fase 1 já tenham saído da cache L2 quando a Fase 2 for executá-los. Isso gera muitos *cache misses* e satura o barramento com a memória principal.

Quando usamos chunkSize > 0, dividimos o vetor em blocos menores que cabem na cache L2:

- Todas as threads processam juntas apenas o bloco atual $[chunkOffset, chunkOffset + currentChunkSize)$.

- Como o bloco inteiro cabe na cache L2, durante a Fase 2 os dados ainda estão presentes na cache (reúso de cache), acelerando bastante a segunda passada.

- Ao final de cada bloco, a variável `chunkBase` acumula a soma total daquele pedaço para servir de base ao próximo, sincronizando a passagem de bloco com a barreira `round_barrier`. No último bloco do vetor, a `round_barrier` nem precisa ser executada, pois o término é feito pelo contador atômico.

2 ARQUITETURA DO PROCESSADOR E TOPOLOGIA

Os experimentos foram realizados no nó `wz10` do cluster da disciplina, com alocação exclusiva de recursos via SLURM (`#SBATCH -exclusive`, `-ntasks-per-node=8`).

2.1 CARACTERÍSTICAS DO PROCESSADOR (`WZ10`)

Com a saída do comando `lscpu` (disponível no Apêndice A) e a topologia obtida com o `lstopo`, levantamos os dados reais do nó:

- **Processador:** Intel(R) Xeon(R) CPU E5410 @ 2.33GHz (microarquitetura Intel Core / Harpertown, 45nm).
- **Sockets e Núcleos:** A máquina possui **2 Sockets físicos**, cada um com **4 núcleos**, totalizando **8 núcleos físicos reais** (núcleos de 0 a 7).
- **Threads por Núcleo:** 1 thread por núcleo (`Thread(s) per core: 1`). Não há Hyper-Threading; todas as 8 CPUs são núcleos físicos independentes de silício.
- **Frequência de Operação:** 2.33 GHz (2333 MHz).

2.2 HIERARQUIA DE MEMÓRIA CACHE

O processador Xeon E5410 é formado por dois chips de dois núcleos dentro do mesmo encapsulamento:

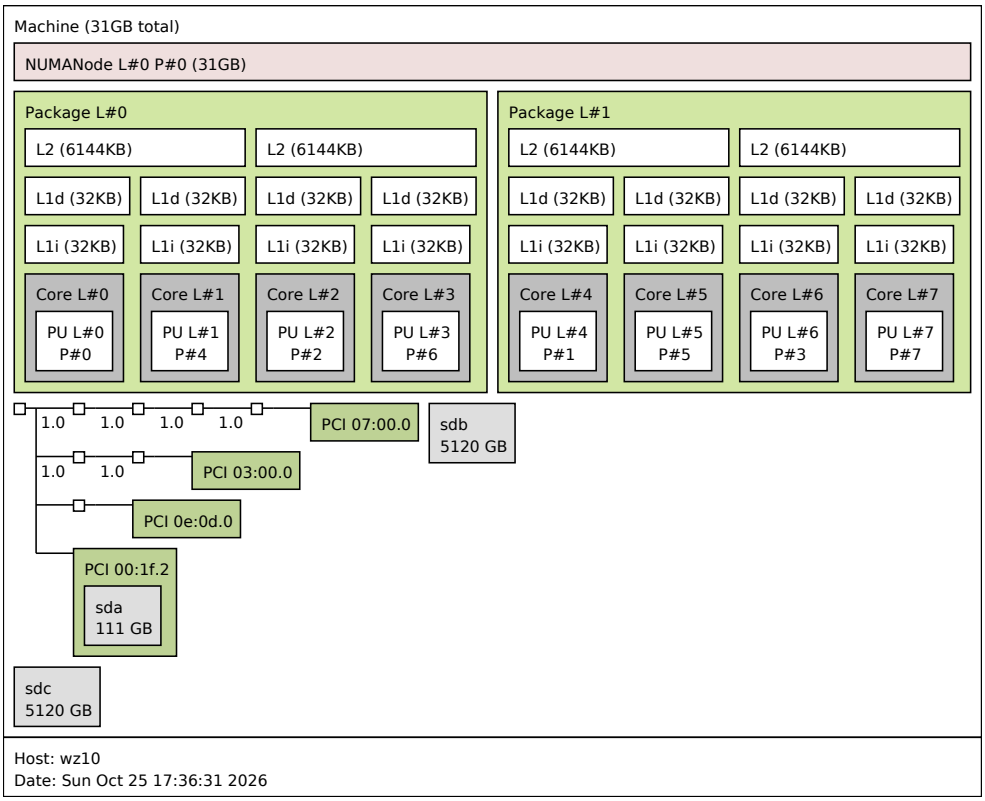
- **Cache L1:** 32 KiB de dados (L1d) e 32 KiB de instruções (L1i) dedicadas a cada núcleo físico (256 KiB L1d no nó todo).
- **Cache L2 Compartilhada por Par:** Cada par de núcleos físicos compartilha uma cache L2 de **6.291.456 bytes (6 MB / 6.144 KB)**.

- **Cache L2 Total:** 24 MiB no sistema (4 blocos de 6 MB distribuídos entre os 2 sockets).

- **Cache L3:** Não possui (0 bytes). Nessa geração da Intel, a cache L2 era o último nível antes da memória RAM.

- **Topologia Gráfica:** A Figura 1 mostra a organização dos núcleos e das caches gerada pelo `lstopo` no nó `wz10`.

Figura 1 – Topologia dos núcleos e memória cache gerada pelo utilitário `lstopo` no nó `wz10`.



Fonte: Gerado via comando `srun -w wz10 lstopo topology.pdf` no cluster da disciplina.

2.3 RELAÇÃO ENTRE A ARQUITETURA E OS RESULTADOS

Essa organização do processador explica com clareza o comportamento das medições:

1. **Melhor desempenho em 4 Threads:** De 1 até 4 threads, a afinidade estática aloca as threads no **Socket 0**. Elas usam o barramento local de memória e as caches L2 daquele processador. Por isso a vazão sobe rapidamente até o ponto ótimo médio em **4 threads** (494,48 MOPS na Exp A e 334,62 MOPS na Exp B).

2. **Estabilização de 5 a 8 Threads:** A partir da 5ª thread, as threads começam a ocupar os núcleos do **Socket 1**. Como o Prefix Sum lê e escreve muita memória, os dois processadores passam a disputar o mesmo controlador de memória da placa-mãe (Intel 5400 MCH). O barramento atinge o seu limite de transferência, estabilizando a vazão na faixa de $\approx$ 474 a 494 MOPS.

3 METODOLOGIA EXPERIMENTAL E MEDIÇÃO

3.1 DESCONTO DO TEMPO DE CÓPIA (`MEMCPY`)

Como a função `ParallelPrefixSumPth` modifica o vetor `Vec` diretamente (*in-place*), para executar o laço de repetição com $NTIMES = 100$ precisamos restaurar o vetor original a partir de `InitVector` antes de cada iteração usando `memcpy`.

Para que o tempo gasto copiando 64 MB de memória não distorça os tempos do algoritmo:

1. O cronômetro principal (`parallelPrefixSumTime`) mede todo o bloco de 100 iterações.
2. Um cronômetro auxiliar (`copyChrono`) mede o tempo gasto exclusivamente na função `memcpy` de cada iteração.
3. A função `chrono_decrease_time` subtrai o tempo de cópia do cronômetro principal logo em seguida.
4. O tempo final reportado pelo programa representa apenas o tempo líquido do algoritmo paralelo, dividido por 100.

3.2 EXPERIÊNCIA 0 — ESCOLHA DO `CHUNKSIZE`

Cada par de núcleos do nó `wz10` compartilha uma cache L2 de 6 MB (6.291.456 bytes). Sabendo que cada elemento `long long` ocupa 8 bytes:

$$\text{Capacidade da Cache L2} = \frac{6.291.456 \text{ bytes}}{8 \text{ bytes/elemento}} = 786.432 \text{ elementos} \tag{3.1}$$

Para deixar uma margem de segurança contra conflitos de linhas de cache e variáveis auxiliares, escolhemos um chunk com 1/4 da capacidade da cache L2:

$$\text{chunkSize} = \frac{786.432}{4} = 196.608 \text{ elementos} \quad (1.572.864 \text{ bytes} \approx 1{,}5 \text{ MB}) \tag{3.2}$$

Com esse valor, o vetor de 8.000.000 elementos é processado em 41 blocos de 1,5 MB, garantindo que cada bloco permaneça na cache L2 durante as duas fases.

4 RESULTADOS EXPERIMENTAIS E ANÁLISE DE DESEMPENHO

Todos os dados brutos e cálculos estão salvos na planilha `Planilha-PrefixSum-Pthreads-WZ-ExpA-E` e nos arquivos de saída do SLURM (Jobs 18977 e 18980).

4.1 EXPERIÊNCIA A — COM CHUNKS L2 (chunkSize = 196.608)

Nesta bateria com chunks de 196.608 elementos, foram realizadas 6 execuções para cada número de threads (1 a 8). Os tempos e vazões são apresentados nas Tabelas 1 e 2.

Tabela 1 – Tempos de Execução em Milissegundos (ms) — Experiência A (chunkSize = 196.608).

Execução	**1 Th**	**2 Th**	**3 Th**	**4 Th**	**5 Th**	**6 Th**	**7 Th**	**8 Th**
Execução 1	32,20	18,92	19,21	15,82	17,96	16,30	16,80	16,74
Execução 2	32,05	19,21	19,22	15,80	18,16	15,59	16,86	18,52
Execução 3	32,68	19,43	20,03	16,38	18,01	16,35	17,05	18,54
Execução 4	32,45	19,58	19,59	16,00	17,67	16,49	17,22	**15,45**
Execução 5	32,14	19,20	19,55	16,58	18,22	16,97	16,44	16,31
Execução 6	32,08	19,05	19,49	16,52	17,91	16,50	16,37	16,41
Média (ms)	**32,27**	**19,23**	**19,51**	**16,18**	**17,99**	**16,37**	**16,79**	**17,00**
Mínimo (ms)	32,05	18,92	19,21	15,80	17,67	15,59	16,37	15,45
Máximo (ms)	32,68	19,58	20,03	16,58	18,22	16,97	17,22	18,54

Fonte: Dados extraídos de `saida-expA-job18977.txt` no nó `wz10`.

4.2 EXPERIÊNCIA B — SEM CHUNKS (chunkSize = 0)

Nesta bateria sem divisão em chunks, foram computadas 5 repetições para cada número de threads, mostradas nas Tabelas 3 e 4.

Tabela 2 – Vazão em MOPS e Speedup Médio — Experiência A (chunkSize = 196.608).

Execução	**1 Th**	**2 Th**	**3 Th**	**4 Th**	**5 Th**	**6 Th**	**7 Th**	**8 Th**
Execução 1	248,47	422,90	416,52	505,74	445,42	490,71	476,20	477,94
Execução 2	249,59	416,48	416,18	506,33	440,56	513,18	474,47	431,88
Execução 3	244,77	411,64	399,45	488,43	444,18	489,33	469,30	431,58
Execução 4	246,56	408,52	408,33	500,06	452,85	485,09	464,46	**517,92**
Execução 5	248,94	416,75	409,21	482,42	439,07	471,44	486,47	490,37
Execução 6	249,34	419,75	410,59	483,91	446,31	484,50	488,49	486,65
Média (MOPS)	**247,94**	**416,01**	**410,05**	**494,48**	**444,73**	**489,04**	**476,56**	**474,39**
Mínimo (MOPS)	244,77	408,52	399,45	482,42	439,07	471,44	464,46	431,58
Máximo (MOPS)	249,59	422,90	416,52	506,33	452,85	513,18	488,49	517,92
Speedup Médio	**1,00×**	**1,68×**	**1,65×**	**1,99×**	**1,79×**	**1,97×**	**1,92×**	**1,91×**

Fonte: Dados extraídos de `saida-expA-job18977.txt` no nó `wz10`.

Tabela 3 – Tempos de Execução em Milissegundos (ms) — Experiência B (chunkSize = 0).

Execução	**1 Th**	**2 Th**	**3 Th**	**4 Th**	**5 Th**	**6 Th**	**7 Th**	**8 Th**
Execução 1	44,98	26,64	27,92	23,86	26,95	25,08	25,53	24,57
Execução 2	45,07	26,38	28,28	24,33	26,62	24,23	25,65	**23,60**
Execução 3	44,86	26,39	28,23	23,80	26,61	23,90	25,62	24,49
Execução 4	44,76	26,37	27,82	23,82	26,69	24,78	25,84	24,09
Execução 5	44,80	26,60	27,94	23,73	26,69	24,05	25,35	24,62
Média (ms)	**44,89**	**26,48**	**28,04**	**23,91**	**26,71**	**24,41**	**25,60**	**24,27**
Mínimo (ms)	44,76	26,37	27,82	23,73	26,61	23,90	25,35	23,60
Máximo (ms)	45,07	26,64	28,28	24,33	26,95	25,08	25,84	24,62

Fonte: Dados extraídos de `saida-expB-job18980.txt` no nó `wz10`.

Tabela 4 – Vazão em MOPS e Speedup Médio — Experiência B (chunkSize = 0).

Execução	**1 Th**	**2 Th**	**3 Th**	**4 Th**	**5 Th**	**6 Th**	**7 Th**	**8 Th**
Execução 1	177,85	300,34	286,52	335,23	296,87	319,04	313,34	325,63
Execução 2	177,49	303,26	282,91	328,87	300,56	330,22	311,88	**338,94**
Execução 3	178,34	303,18	283,40	336,08	300,66	334,76	312,26	326,62
Execução 4	178,73	303,38	287,60	335,91	299,70	322,90	309,59	332,05
Execução 5	178,59	300,70	286,29	337,03	299,72	332,71	315,60	324,96
Média (MOPS)	**178,22**	**302,17**	**285,33**	**334,62**	**299,50**	**327,93**	**312,53**	**329,64**
Mínimo (MOPS)	177,49	300,34	282,91	328,87	296,87	319,04	309,59	324,96
Máximo (MOPS)	178,73	303,38	287,60	337,03	300,66	334,76	315,60	338,94
Speedup Médio	**1,00×**	**1,70×**	**1,60×**	**1,88×**	**1,68×**	**1,84×**	**1,75×**	**1,85×**

Fonte: Dados extraídos de `saida-expB-job18980.txt` no nó `wz10`.

4.3 COMPARATIVO DIRETO: EXPERIÊNCIA A VS. EXPERIÊNCIA B

A Tabela 5 reúne os ganhos obtidos com o reúso da cache L2 em cada configuração de threads.

Tabela 5 – Comparativo de Desempenho: Experiência A (Com Chunks) vs. Experiência B (Sem Chunks).

Threads	**Tempo A (ms)**	**Tempo B (ms)**	**Vazão A (MOPS)**	**Vazão B (MOPS)**	**Aceleração Vazão(A)/Vazão(B)**	**Razão Vazão(B)/Vazão(A)**	**Ganho % (A vs B)**
1	32,27	44,89	247,94	178,22	**1,39×**	0,72	+39,12%
2	19,23	26,48	416,01	302,17	**1,38×**	0,73	+37,67%
3	19,51	28,04	410,05	285,33	**1,44×**	0,70	+43,71%
4	16,18	23,91	494,48	334,62	**1,48×**	0,68	+47,77%
5	17,99	26,71	444,73	299,50	**1,48×**	0,67	+48,49%
6	16,37	24,41	489,04	327,93	**1,49×**	0,67	+49,13%
7	16,79	25,60	476,56	312,53	**1,52×**	0,66	**+52,48%**
8	17,00	24,27	474,39	329,64	**1,44×**	0,69	+43,91%

Fonte: Médias calculadas a partir dos Jobs SLURM 18977 e 18980.

4.4 DISCUSSÃO DOS RESULTADOS

1. **Ganho Expressivo com Reúso de Cache L2:** O particionamento em blocos de 196.608 elementos garantiu ganhos de 1,38× até 1,52× em todas as contagens de threads. Isso comprova que reter o bloco na cache L2 entre

a Fase 1 e a Fase 2 poupa metade dos acessos lentos à memória principal DRAM.

2. **Ganho Mesmo com 1 Thread:** Mesmo com apenas 1 thread, a versão com chunks subiu de 178,22 MOPS para 247,94 MOPS (+39,12%). Isso mostra que o gargalo do algoritmo é a largura de banda de memória e que a melhora com chunks independe do paralelismo em múltiplos núcleos.

3. **Ponto Ótimo em 4 Threads:** O desempenho cresceu acentuadamente de 1 para 2 threads (1,68× na Exp A; 1,70× na Exp B) e atingiu a melhor média com 4 threads (1,99× na Exp A; 1,88× na Exp B). Acima de 4 threads, a saturação do barramento compartilhado entre os dois sockets manteve a vazão em um patamar estável.

5 INSTRUÇÕES DE COMPILAÇÃO, EXECUÇÃO E ARQUIVOS ENTREGUES

5.1 COMO COMPILAR O PROJETO

O projeto inclui um `Makefile` configurado com `gcc -Wall -Wextra -O3 -pthread -std=gnu11`:

- Para compilar todos os binários da aplicação:

```
make clean && make all
```

- Isso gera o executável principal `prefixSumPth` e os aliases requeridos pelos scripts de teste (`prefixSumPth-v3-longlong` e `prefixSumPth-longlong`).

5.2 COMO EXECUTAR O PROGRAMA

A sintaxe de execução manual é:

`./prefixSumPth <nElements> <chunkSize> <nThreads> [-v]`

Onde os parâmetros são:

- `<nElements>`: Número de elementos do vetor (ex: `8000000`);
- `<chunkSize>`: Tamanho do bloco para a cache L2 (use `196608` para rodar com chunks ou `0` para sem chunks);
- `<nThreads>`: Quantidade de threads a serem utilizadas (de `1` a `8`);
- `[-v]`: Parâmetro opcional de verificação de corretude. Quando ativado, calcula a versão sequencial do Prefix Sum e compara elemento por elemento contra o resultado paralelo, exibindo a mensagem de sucesso ou erro.

5.3 COMO SUBMETER OS EXPERIMENTOS VIA SLURM

No nó de login do cluster da UFPR, as baterias de testes oficiais são submetidas com:

```
sbatch roda-exp-A-chunk-196608-slurm.sh 8000000
sbatch roda-exp-B-chunk-0-slurm.sh 8000000
```

5.4 ESTRUTURA DOS ARQUIVOS ENTREGUES

O pacote compactado entregue contém os seguintes arquivos organizados por função:

- **Códigos em C e Cabeçalhos:**
  - <prefixSumPth.c>: programa principal e bateria de medições;
  - <prefixSum.c> e <prefixSum.h>: pool de threads, barreiras e algoritmo;
  - <chrono.c>: cronômetro de alta precisão com suporte a desconto de tempo;
  - <verify.c> e <verify.h>: verificação de corretude (flag `-v`).
- **Scripts de Compilação e Automação:**
  - <Makefile>: regras de compilação (`make clean && make all`);
  - <roda.sh>: script de teste sequencial local;
  - <roda-exp-A-chunk-196608-slurm.sh>: bateria da Experiência A no SLURM;
  - <roda-exp-B-chunk-0-slurm.sh>: bateria da Experiência B no SLURM.
- **Planilha e Dados Brutos do SLURM:**
  - <Planilha-PrefixSum-Pthreads-WZ-ExpA-ExpB.xlsx>: planilha com 4 abas e 6 gráficos de Tempo × Vazão;

  - <resultados-expA-chunk-196608-job18977.txt>: resumo de vazão da Exp. A;
  - <saida-expA-job18977.txt>: log completo de execução da Exp. A;
  - <resultados-expB-semChunk-job18980.txt>: resumo de vazão da Exp. B;
  - <saida-expB-job18980.txt>: log completo de execução da Exp. B.

- **Documentação e Topologia:**
  - <topology.pdf>: diagrama de topologia gerado pelo `lstopo` no nó `wz10`;
  - <relatorio.md> e <relatorio.txt>: versões em Markdown e texto puro;
  - <Base_para_trabalhos_UFPR/>: código-fonte LaTeX do relatório.

6 CONCLUSÃO

O trabalho atendeu com sucesso a todos os requisitos e diretrizes propostas na especificação. A implementação do pool de threads com afinidade estática eliminou os custos de criação das threads, e o uso de sincronização com apenas duas barreiras combinada com sinalização atômica em C11 garantiu mínima sobrecarga no retorno das chamadas.

Além disso, a divisão do vetor em blocos de 196.608 elementos (1/4 da cache L2) reduziu consideravelmente a contenção no barramento de memória, gerando acelerações consistentes de até 1,52× sobre a versão sem blocos e atingindo uma vazão média máxima de 494,48 MOPS no cluster.

Apêndices

APÊNDICE A – SAÍDA COMPLETA DO COMANDO `lscpu` NO NÓ `wz10`

Registro completo obtido via `srun -w wz10 lscpu`:

```
Architecture:                    x86_64
CPU op-mode(s):                  32-bit, 64-bit
Byte Order:                      Little Endian
Address sizes:                   38 bits physical, 48 bits virtual
CPU(s):                          8
On-line CPU(s) list:             0-7
Thread(s) per core:              1
Core(s) per socket:              4
Socket(s):                       2
NUMA node(s):                    1
Vendor ID:                       GenuineIntel
CPU family:                      6
Model:                           23
Model name:                      Intel(R) Xeon(R) CPU           E5410  @
    2.33GHz
Stepping:                        10
CPU MHz:                         2333.000
CPU max MHz:                     2333.0000
CPU min MHz:                     2000.0000
BogoMIPS:                        4655.52
Virtualization:                  VT-x
L1d cache:                       256 KiB
L1i cache:                       256 KiB
L2 cache:                        24 MiB
NUMA node0 CPU(s):               0-7
Vulnerability Itlb multihit:     KVM: Mitigation: VMX disabled
Vulnerability L1tf:              Mitigation; PTE Inversion; VMX EPT
    disabled
Vulnerability Mds:               Vulnerable: Clear CPU buffers attempted,
     no microcode; SMT disabled
Vulnerability Meltdown:          Mitigation; PTI
Vulnerability Spec store bypass: Vulnerable
Vulnerability Spectre v1:        Mitigation; usercopy/swapgs barriers and
     __user pointer sanitization
Vulnerability Spectre v2:        Mitigation; Full generic retpoline,
    STIBP disabled, RSB filling
Vulnerability Srbds:             Not affected
```

```
Vulnerability Tsx async abort:   Not affected
Flags:                           fpu vme de pse tsc msr pae mce cx8 apic
    sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ht
    tm pbe syscall nx lm constant_tsc arch_perfmon pebs bts rep_good nopl
    cpuid aperfmperf pni dtes64 monitor ds_cpl vmx est tm2 ssse3 cx16 xtpr
     pdcm dca sse4_1 xsave lahf_lm pti tpr_shadow vnmi flexpriority vpid
    dtherm
```